Ahsanullah University of Science and Technology
Department of Computer Science and Engineering

CSE 3104 Project Proposal

Topic: Advanced Hospital Management System

By
Student ID: 20230104110
Student ID: 20230104119
Student ID: 20230104122

1. Introduction

Healthcare institutions manage large volumes of sensitive, interrelated data including patient demographics, clinical histories, admissions, prescriptions, laboratory processes, staff information, and inventory records. A Hospital Management System (HMS) is a web-based information system designed to coordinate these operational and clinical workflows in a secure, consistent, and efficient manner. This project proposes the development of an **Advanced Hospital Management System** backed by a **relational SQL database** and implemented with a modern web architecture (frontend + RESTful backend + database).

A relational database is appropriate for hospital environments because hospital data exhibits strong structural relationships. For example, a department employs many doctors; a doctor attends multiple appointments; a patient can have multiple admissions and medical records over time; and prescriptions must reference both the patient and the issuing doctor. By modeling these entities and relationships using an Entity Relationship Diagram (ERD) and then implementing them using SQL tables, constraints, and transactions, the system ensures **data integrity**, **consistency**, and **reliable multi-user concurrency**.

Additionally, hospitals require **controlled access** to medical data. Therefore, the system includes **Role-Based Access Control (RBAC)** so that administrators, doctors, nurses, and

patients only see and modify the information relevant to their roles. Administrative users will have comprehensive permissions, including approval or rejection of staff applications for doctor and nurse accounts. Through careful database design and secure API development, this HMS supports both daily operations and long-term analytics such as patient flow reporting, department workload measurement, and resource availability monitoring (ICU/CCU/NCU beds, blood inventory, etc.).

---

2. Motivation

Hospital systems demand high reliability and correctness because errors can negatively affect patient safety and institutional operations. SQL-based relational databases are widely used in enterprise healthcare systems because they provide strong guarantees and mature tooling. The motivations for selecting an SQL-based approach include:

2.1 ACID Compliance for Critical Medical Data

Medical and administrative operations frequently require multiple steps that must either complete fully or not execute at all. SQL databases support **ACID properties** (Atomicity, Consistency, Isolation, Durability), enabling the system to handle critical workflows such as appointment booking, admission creation, prescription issuance, and blood request processing without partial or inconsistent updates.

2.2 Structured Relationships and Complex Data Modeling

Healthcare information is inherently relational. SQL supports explicit relationships using primary keys and foreign keys, enabling the accurate modeling of real-world constraints such as:

- One department can have many doctors.
- One doctor can manage many admitted patients.
- Patients can have multiple admissions and medical records.
- Prescriptions may include multiple medications.

2.3 Data Integrity through Constraints

SQL databases enforce integrity rules through:

- **Foreign key constraints** (prevent orphan records)

- **Unique constraints** (e.g., license numbers, emails)
- **NOT NULL constraints** (mandatory fields)
- **Check constraints** (valid status values)

These features reduce data anomalies and ensure consistent records across the system.

2.4 Industry Standard for Enterprise Applications

MySQL and PostgreSQL are widely supported, scalable, and compatible with many frameworks. Their ecosystem includes mature administration tools, monitoring, backup solutions, and performance tuning capabilities. This makes SQL a practical and professional choice for long-term hospital deployments.

2.5 Reporting, Analytics, and Query Efficiency

Hospitals need operational analytics such as:

- department-wise patient load,
- doctor workload,
- admission trends,
- ICU/CCU/NCU bed availability,
- blood inventory summaries.

SQL supports JOIN queries, aggregations, views, indexing, and query planning, providing superior reporting capability compared to unstructured storage solutions.

---

3. Requirements

3.1 Goals

- Develop a secure and scalable **web-based** hospital management system using a **normalized SQL database**.

- Implement **Role-Based Access Control (RBAC)** with role-specific operations and permissions.
- Ensure **data integrity** through primary keys, foreign keys, constraints, and validation rules.
- Use **ACID-compliant transactions** for sensitive multi-step operations (admissions, appointments, prescriptions, inventory updates).
- Provide facilities for advanced hospital operations including **Blood Bank management** and **ICU/CCU/NCU availability tracking**.

3.2 Objectives

- Design a relational schema in **Third Normal Form (3NF)** to minimize redundancy and update anomalies.
- Create RESTful APIs integrating SQL database operations for all major modules.
- Build role-specific interfaces for:
    - Admin: system management + approval of staff applications
    - Doctor: appointments, admissions, prescriptions, medical record entries
    - Nurse: ward/bed coordination, patient monitoring workflows (as permitted)
    - Patient: view profile, appointments, admissions summary, prescriptions, blood requests
- Implement stored procedures or transaction blocks for operations requiring atomicity.
- Implement audit and logging mechanisms using triggers or application logs for sensitive changes.

3.3 Functionalities

3.3.1 User and Role Management (RBAC)

- User registration and authentication (email + password hash).
- Role assignment: Admin, Doctor, Nurse, Patient.
- Role-specific access restrictions for APIs and UI components.

3.3.2 Admin Module

- Full access to system data across modules (superuser privileges).
- View and manage departments, doctors, nurses, patients, admissions, beds, blood bank, and inventory.
- **Approve or reject Staff Applications** (doctor/nurse job applicants).
- Create and manage hospital units (Ward Units: ICU/CCU/NCU).

3.3.3 Patient Management

- Store demographic data and emergency contact information.
- Maintain longitudinal medical record history.
- Support multiple admissions over time per patient.

3.3.4 Doctor Management

- Maintain doctor profiles (specialization, license number).
- Each doctor belongs to exactly one department (mandatory relationship).
- View and manage assigned admissions and appointments.

3.3.5 Appointment Scheduling

- Book appointments between patients and doctors.
- Track appointment statuses (Scheduled, Completed, Cancelled, No-Show).
- Use transactions to prevent double booking and concurrency issues.

3.3.6 Admission Management

- Admission records store the department under which the patient is admitted.
- Each admission is assigned exactly one doctor.
- Admission may involve bed assignment for ICU/CCU/NCU as needed.

3.3.7 Medical Records

- Medical records are maintained per patient with diagnosis and treatment history.
- Supports multiple entries per patient over time.

3.3.8 Prescriptions

- A doctor issues prescriptions for a patient.
- Prescription contains medication details and dosage instructions.
- (Optional advanced design) prescription includes multiple medicines using line items.

3.3.9 Blood Bank Management

- Blood bank stores inventory by blood group and component type.
- Donor records maintained with blood group and last donation date.
- Patients can request blood; requests are processed and fulfilled by blood bank.

3.3.10 ICU/CCU/NCU Bed Availability

- Ward units categorized as ICU, CCU, or NCU.
- Beds belong to a ward unit and have status:
    - Available / Occupied / Maintenance
- Availability is determined by querying bed status per unit type.

3.4 Features

3.4.1 SQL Security Features

- Parameterized queries and prepared statements to prevent SQL injection.
- Password hashing (bcrypt/argon2) and secure authentication sessions or JWT tokens.
- Database roles and privileges (optional advanced layer) aligned with application RBAC.

3.4.2 Strong Data Relationships and Integrity

- Explicit one-to-one and one-to-many relationships modeled in ERD.
- Foreign keys ensure all references remain valid (e.g., admissions must reference an existing patient and doctor).
- Unique constraints enforce identity fields such as doctor license numbers and user emails.

3.4.3 Transaction Management

- Appointment booking and bed assignment operations require atomic updates.
- Use BEGIN/COMMIT/ROLLBACK to guarantee correctness under concurrent usage.

3.4.4 Reporting and Analytics

- SQL queries for:
    - doctor workload,
    - department admissions per month,
    - ICU bed occupancy,
    - blood availability per group,
    - patient history views using JOIN.

---

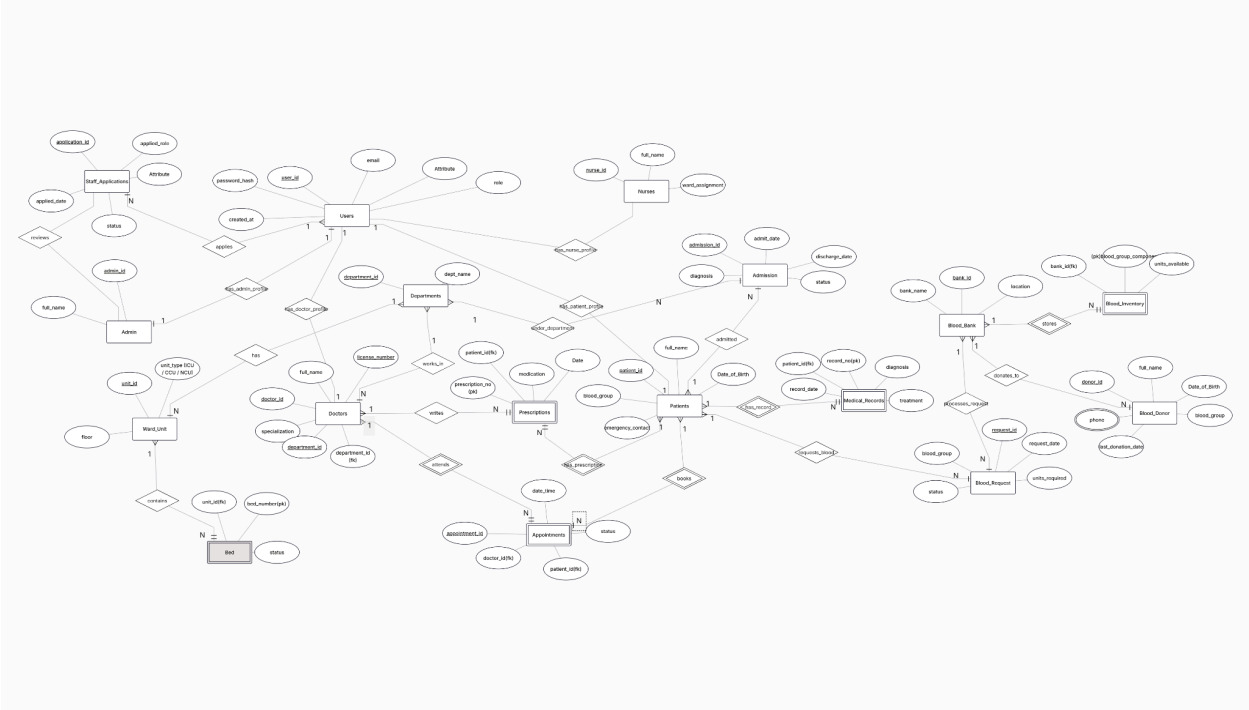
4. Entity Relation Diagram (Chen Notation)

The ERD is designed using Chen notation (rectangles for entities, ovals for attributes, diamonds for relationships). Both strong and weak entities may be used where appropriate. The following describes the ERD structure in implementation-ready form.

4.1 Main Entities and Key Attributes

Users *(user_id PK, email UNIQUE, password_hash, role, created_at)*
Admin *(admin_id PK, full_name, phone)*
Patients *(patient_id PK, full_name, dob, blood_group, emergency_contact)*
Doctors *(doctor_id PK, full_name, specialization, license_number UNIQUE)*
Nurses *(nurse_id PK, full_name, ward_assignment, shift_timing)*
Departments *(department_id PK, dept_name UNIQUE, location, phone_ext)*
Appointment *(appointment_id PK, date_time, status)*
Admission *(admission_id PK, admit_date, discharge_date, diagnosis, status)*
Medical_Records *(record_no partial key, diagnosis, treatment, record_date)*
Prescriptions *(prescription_no partial key, medication, dosage, date)*
Blood_Bank *(bank_id PK, bank_name, location, phone)*
Blood_Inventory *(inventory_item partial key, blood_group, component_type, units_available, last_updated)*
Blood_Donor *(donor_id PK, full_name, dob, blood_group, phone, last_donation_date)*
Blood_Request *(request_id PK, request_date, blood_group, component_type, units_required, status)*
Ward_Unit *(unit_id PK, unit_type, floor, contact_ext)*
Bed *(bed_number partial key, status, daily_rate)*
Staff_Applications *(application_id PK, applied_role, status, applied_date, decision_date,*

remarks)



4.2 Core Relationships and Cardinalities

1. **Users 1:1 Patients/Doctors/Nurses/Admin** (profile relationship)

2. **Departments 1:N Doctors** (each doctor belongs to exactly one department)

3. **Patients 1:N Appointment** and **Doctors 1:N Appointment** (appointments require both)

4. **Patients 1:N Admission**, **Departments 1:N Admission**, **Doctors 1:N Admission**

    - Each admission belongs to one patient, one department, and one assigned doctor

5. **Patients 1:N Medical_Records** *(identifying relationship if modeled weak)*

6. **Patients 1:N Prescriptions** *(identifying relationship if modeled weak)* and **Doctors 1:N Prescriptions**

7. **Blood_Bank 1:N Blood_Inventory** *(identifying relationship if modeled weak)*

8. **Blood_Donor N:1 Blood_Bank**

9. **Patients 1:N Blood_Request** and **Blood_Bank 1:N Blood_Request**

10. **Ward_Unit 1:N Bed** *(identifying relationship if modeled weak)*

11. **Admin 1:N Staff_Applications** (Admin reviews/approves/rejects applications)

4.3 Note on Foreign Keys (Implementation Mapping)

In the relational database, one-to-many relationships are implemented by placing the foreign key on the "many" side. For example:

- `Doctors.department_id` references `Departments.department_id`
- `Admission.patient_id`, `Admission.department_id`, and `Admission.doctor_id` reference their parent tables
- `Staff_Applications.reviewed_by_admin_id` references `Admin.admin_id`

---

5. Development Tools and Platforms

5.1 Database Layer

- **MySQL 8.0** or **PostgreSQL 14+** as the relational database engine.
- Supports constraints, indexing, transactions, and stored procedures (dialect-dependent).

5.2 Backend

- **Node.js + Express** for RESTful API development.
- Database drivers:
    - MySQL: `mysql2`
    - PostgreSQL: `pg`
- Optional ORM:

    - Sequelize (Node.js)
- Authentication:
    - bcrypt/argon2 password hashing
    - JWT or secure sessions
- API practices:
    - Proper HTTP status codes
    - Validation middleware
    - Error handling and logging

5.3 Frontend

- **React.js** for UI development
- **Axios** for API calls
- Role-based UI rendering:
    - Admin dashboard
    - Doctor portal
    - Nurse portal
    - Patient portal

5.4 Testing and Documentation Tools

- **Postman** for API testing
- **Git** for version control

5.5 Database Modeling Tools

- **ERDPlus.com** for Chen ER diagram creation
- MySQL Workbench / pgAdmin for schema inspection and management